

```
pip install ucimlrepo
```

```
Collecting ucimlrepo
```

```
  Downloading ucimlrepo-0.0.7-py3-none-any.whl.metadata (5.5 kB)
```

```
Requirement already satisfied: pandas>=1.0.0 in  
/usr/local/lib/python3.13/dist-packages (from ucimlrepo) (2.2.3)
```

```
Requirement already satisfied: certifi>=2020.12.5 in  
/usr/local/lib/python3.13/dist-packages (from ucimlrepo) (2026.7.22)
```

```
Requirement already satisfied: numpy>=1.26.0 in  
/usr/local/lib/python3.13/dist-packages (from pandas>=1.0.0-  
>ucimlrepo) (2.1.3)
```

```
Requirement already satisfied: python-dateutil>=2.8.2 in  
/usr/local/lib/python3.13/dist-packages (from pandas>=1.0.0-  
>ucimlrepo) (2.9.0.post0)
```

```
Requirement already satisfied: pytz>=2020.1 in  
/usr/local/lib/python3.13/dist-packages (from pandas>=1.0.0-  
>ucimlrepo) (2025.2)
```

```
Requirement already satisfied: tzdata>=2022.7 in  
/usr/local/lib/python3.13/dist-packages (from pandas>=1.0.0-  
>ucimlrepo) (2026.3)
```

```
Requirement already satisfied: six>=1.5 in  
/usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.2-  
>pandas>=1.0.0->ucimlrepo) (1.17.0)
```

```
Downloading ucimlrepo-0.0.7-py3-none-any.whl (8.0 kB)
```

```
Installing collected packages: ucimlrepo
```

```
Successfully installed ucimlrepo-0.0.7
```

```
from ucimlrepo import fetch_ucirepo
```

```
# Fetch dataset
```

```
breast_cancer_wisconsin_diagnostic = fetch_ucirepo(id=17)
```

```
# Data as pandas DataFrames
```

```
X = breast_cancer_wisconsin_diagnostic.data.features
```

```
y = breast_cancer_wisconsin_diagnostic.data.targets
```

```
# Display first 5 feature records
```

```
print("Features:")
```

```
print(X.head())
```

```
# Display first 5 target records
```

```
print("\nTarget:")
```

```
print(y.head())
```

```
# Metadata
```

```
print("\nMetadata:")
```

```
print(breast_cancer_wisconsin_diagnostic.metadata)
```

```
# Variable information
```

```
print("\nVariables:")
print(breast_cancer_wisconsin_diagnostic.variables)
```

Features:

	radius1	texture1	perimeter1	area1	smoothness1	compactness1	\
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	
3	11.42	20.38	77.58	386.1	0.14250	0.28390	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	

	concavity1	concave_points1	symmetry1	fractal_dimension1	...
radius3 \					
0	0.3001	0.14710	0.2419	0.07871	...
25.38					
1	0.0869	0.07017	0.1812	0.05667	...
24.99					
2	0.1974	0.12790	0.2069	0.05999	...
23.57					
3	0.2414	0.10520	0.2597	0.09744	...
14.91					
4	0.1980	0.10430	0.1809	0.05883	...
22.54					

	texture3	perimeter3	area3	smoothness3	compactness3	concavity3
\						
0	17.33	184.60	2019.0	0.1622	0.6656	0.7119
1	23.41	158.80	1956.0	0.1238	0.1866	0.2416
2	25.53	152.50	1709.0	0.1444	0.4245	0.4504
3	26.50	98.87	567.7	0.2098	0.8663	0.6869
4	16.67	152.20	1575.0	0.1374	0.2050	0.4000

	concave_points3	symmetry3	fractal_dimension3
0	0.2654	0.4601	0.11890
1	0.1860	0.2750	0.08902
2	0.2430	0.3613	0.08758
3	0.2575	0.6638	0.17300
4	0.1625	0.2364	0.07678

[5 rows x 30 columns]

Target:

	Diagnosis
0	M
1	M

2	M
3	M
4	M

Metadata:

```
{'uci_id': 17, 'name': 'Breast Cancer Wisconsin (Diagnostic)',
'repository_url':
'https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic', 'data_url':
'https://archive.ics.uci.edu/static/public/17/data.csv', 'abstract':
'Diagnostic Wisconsin Breast Cancer Database.', 'area': 'Health and
Medicine', 'tasks': ['Classification'], 'characteristics':
['Multivariate'], 'num_instances': 569, 'num_features': 30,
'feature_types': ['Real'], 'demographics': [], 'target_col':
'Diagnosis', 'index_col': ['ID'], 'has_missing_values': 'no',
'missing_values_symbol': None, 'year_of_dataset_creation': 1993,
'last_updated': 'Fri Nov 03 2023', 'dataset_doi': '10.24432/C5DW2B',
'creators': ['William Wolberg', 'Olvi Mangasarian', 'Nick Street', 'W.
Street'], 'intro_paper': {'ID': 230, 'type': 'NATIVE', 'title':
'Nuclear feature extraction for breast tumor diagnosis', 'authors':
'W. Street, W. Wolberg, O. Mangasarian', 'venue': 'Electronic
imaging', 'year': 1993, 'journal': None, 'DOI': '10.1117/12.148698',
'URL':
'https://www.semanticscholar.org/paper/53f0fbb425bc14468eb3bf96b2e1d41
ba8087f36', 'sha': None, 'corpus': None, 'arxiv': None, 'mag': None,
'acl': None, 'pmid': None, 'pmcid': None}, 'additional_info':
{'summary': 'Features are computed from a digitized image of a fine
needle aspirate (FNA) of a breast mass. They describe characteristics
of the cell nuclei present in the image. A few of the images can be
found at http://www.cs.wisc.edu/~street/images/\r\n\r\nSeparating
plane described above was obtained using Multisurface Method-Tree
(MSM-T) [K. P. Bennett, "Decision Tree Construction Via Linear
Programming." Proceedings of the 4th Midwest Artificial Intelligence
and Cognitive Science Society, pp. 97-101, 1992], a classification
method which uses linear programming to construct a decision tree.
Relevant features were selected using an exhaustive search in the
space of 1-4 features and 1-3 separating planes.\r\n\r\nThe actual
linear program used to obtain the separating plane in the 3-
dimensional space is that described in: [K. P. Bennett and O. L.
Mangasarian: "Robust Linear Programming Discrimination of Two Linearly
Inseparable Sets", Optimization Methods and Software 1, 1992, 23-34].\r\n\r\n\r\nThis database is also available through the UW CS ftp server:\r\n\r\nftp ftp.cs.wisc.edu\r\nncd
math-prog/cpo-dataset/machine-learn/WDBC/', 'purpose': None,
'funded_by': None, 'instances_represent': None,
'recommended_data_splits': None, 'sensitive_data': None,
'preprocessing_description': None, 'variable_info': '1) ID number\r\n
2) Diagnosis (M = malignant, B = benign)\r\n3-32)\r\n\r\nTen real-
valued features are computed for each cell nucleus:\r\n\r\n(a) radius
(mean of distances from center to points on the perimeter)\r\n\r\n(b)
```

texture (standard deviation of gray-scale values)\r\n\tc) perimeter\r\n\t d) area\r\n\t e) smoothness (local variation in radius lengths)\r\n\t f) compactness (perimeter^2 / area - 1.0)\r\n\t g) concavity (severity of concave portions of the contour)\r\n\t h) concave points (number of concave portions of the contour)\r\n\t i) symmetry \r\n\t j) fractal dimension ("coastline approximation" - 1)', 'citation': None}}

Variables:

	name	role	type	demographic	description
units \					
0	ID	ID	Categorical	None	None
None					
1	Diagnosis	Target	Categorical	None	None
None					
2	radius1	Feature	Continuous	None	None
None					
3	texture1	Feature	Continuous	None	None
None					
4	perimeter1	Feature	Continuous	None	None
None					
5	area1	Feature	Continuous	None	None
None					
6	smoothness1	Feature	Continuous	None	None
None					
7	compactness1	Feature	Continuous	None	None
None					
8	concavity1	Feature	Continuous	None	None
None					
9	concave_points1	Feature	Continuous	None	None
None					
10	symmetry1	Feature	Continuous	None	None
None					
11	fractal_dimension1	Feature	Continuous	None	None
None					
12	radius2	Feature	Continuous	None	None
None					
13	texture2	Feature	Continuous	None	None
None					
14	perimeter2	Feature	Continuous	None	None
None					
15	area2	Feature	Continuous	None	None
None					
16	smoothness2	Feature	Continuous	None	None
None					
17	compactness2	Feature	Continuous	None	None
None					
18	concavity2	Feature	Continuous	None	None
None					
19	concave_points2	Feature	Continuous	None	None
None					

20	symmetry2	Feature	Continuous	None	None
None					
21	fractal_dimension2	Feature	Continuous	None	None
None					
22	radius3	Feature	Continuous	None	None
None					
23	texture3	Feature	Continuous	None	None
None					
24	perimeter3	Feature	Continuous	None	None
None					
25	area3	Feature	Continuous	None	None
None					
26	smoothness3	Feature	Continuous	None	None
None					
27	compactness3	Feature	Continuous	None	None
None					
28	concavity3	Feature	Continuous	None	None
None					
29	concave_points3	Feature	Continuous	None	None
None					
30	symmetry3	Feature	Continuous	None	None
None					
31	fractal_dimension3	Feature	Continuous	None	None
None					
missing_values					
0	no				
1	no				
2	no				
3	no				
4	no				
5	no				
6	no				
7	no				
8	no				
9	no				
10	no				
11	no				
12	no				
13	no				
14	no				
15	no				
16	no				
17	no				
18	no				
19	no				
20	no				
21	no				
22	no				
23	no				

```

24         no
25         no
26         no
27         no
28         no
29         no
30         no
31         no

```

```

# Summary statistics for all features
print(X.describe())

```

	radius1	texture1	perimeter1	area1	smoothness1 \
count	569.000000	569.000000	569.000000	569.000000	569.000000
mean	14.127292	19.289649	91.969033	654.889104	0.096360
std	3.524049	4.301036	24.298981	351.914129	0.014064
min	6.981000	9.710000	43.790000	143.500000	0.052630
25%	11.700000	16.170000	75.170000	420.300000	0.086370
50%	13.370000	18.840000	86.240000	551.100000	0.095870
75%	15.780000	21.800000	104.100000	782.700000	0.105300
max	28.110000	39.280000	188.500000	2501.000000	0.163400

	compactness1	concavity1	concave_points1	symmetry1 \
count	569.000000	569.000000	569.000000	569.000000
mean	0.104341	0.088799	0.048919	0.181162
std	0.052813	0.079720	0.038803	0.027414
min	0.019380	0.000000	0.000000	0.106000
25%	0.064920	0.029560	0.020310	0.161900
50%	0.092630	0.061540	0.033500	0.179200
75%	0.130400	0.130700	0.074000	0.195700
max	0.345400	0.426800	0.201200	0.304000

	fractal_dimension1	...	radius3	texture3	perimeter3 \
count	569.000000	...	569.000000	569.000000	569.000000
mean	0.062798	...	16.269190	25.677223	107.261213
std	0.007060	...	4.833242	6.146258	33.602542
min	0.049960	...	7.930000	12.020000	50.410000
25%	0.057700	...	13.010000	21.080000	84.110000
50%	0.061540	...	14.970000	25.410000	97.660000
75%	0.066120	...	18.790000	29.720000	125.400000
max	0.097440	...	36.040000	49.540000	251.200000

	area3	smoothness3	compactness3	concavity3
concave_points3 \				
count	569.000000	569.000000	569.000000	569.000000
mean	880.583128	0.132369	0.254265	0.272188
std	569.356993	0.022832	0.157336	0.208624
min	0.114606			
max	0.065732			

min	185.200000	0.071170	0.027290	0.000000
0.000000				
25%	515.300000	0.116600	0.147200	0.114500
0.064930				
50%	686.500000	0.131300	0.211900	0.226700
0.099930				
75%	1084.000000	0.146000	0.339100	0.382900
0.161400				
max	4254.000000	0.222600	1.058000	1.252000
0.291000				

	symmetry3	fractal_dimension3
count	569.000000	569.000000
mean	0.290076	0.083946
std	0.061867	0.018061
min	0.156500	0.055040
25%	0.250400	0.071460
50%	0.282200	0.080040
75%	0.317900	0.092080
max	0.663800	0.207500

[8 rows x 30 columns]

```
print("Mean:")
print(X.mean())
```

```
print("\nMedian:")
print(X.median())
```

```
print("\nStandard Deviation:")
print(X.std())
```

```
Mean:
radius1          14.127292
texture1         19.289649
perimeter1       91.969033
area1            654.889104
smoothness1      0.096360
compactness1     0.104341
concavity1       0.088799
concave_points1  0.048919
symmetry1        0.181162
fractal_dimension1 0.062798
radius2          0.405172
texture2         1.216853
perimeter2       2.866059
area2            40.337079
smoothness2      0.007041
compactness2     0.025478
concavity2       0.031894
```

concave_points2	0.011796
symmetry2	0.020542
fractal_dimension2	0.003795
radius3	16.269190
texture3	25.677223
perimeter3	107.261213
area3	880.583128
smoothness3	0.132369
compactness3	0.254265
concavity3	0.272188
concave_points3	0.114606
symmetry3	0.290076
fractal_dimension3	0.083946

dtype: float64

Median:

radius1	13.370000
texture1	18.840000
perimeter1	86.240000
area1	551.100000
smoothness1	0.095870
compactness1	0.092630
concavity1	0.061540
concave_points1	0.033500
symmetry1	0.179200
fractal_dimension1	0.061540
radius2	0.324200
texture2	1.108000
perimeter2	2.287000
area2	24.530000
smoothness2	0.006380
compactness2	0.020450
concavity2	0.025890
concave_points2	0.010930
symmetry2	0.018730
fractal_dimension2	0.003187
radius3	14.970000
texture3	25.410000
perimeter3	97.660000
area3	686.500000
smoothness3	0.131300
compactness3	0.211900
concavity3	0.226700
concave_points3	0.099930
symmetry3	0.282200
fractal_dimension3	0.080040

dtype: float64

Standard Deviation:

```
radius1          3.524049
texture1         4.301036
perimeter1      24.298981
area1           351.914129
smoothness1     0.014064
compactness1    0.052813
concavity1      0.079720
concave_points1 0.038803
symmetry1       0.027414
fractal_dimension1 0.007060
radius2          0.277313
texture2         0.551648
perimeter2       2.021855
area2           45.491006
smoothness2     0.003003
compactness2    0.017908
concavity2      0.030186
concave_points2 0.006170
symmetry2       0.008266
fractal_dimension2 0.002646
radius3          4.833242
texture3         6.146258
perimeter3      33.602542
area3           569.356993
smoothness3     0.022832
compactness3    0.157336
concavity3      0.208624
concave_points3 0.065732
symmetry3       0.061867
fractal_dimension3 0.018061
dtype: float64
```

```
# Check missing values in each feature
```

```
print("Missing values:")
```

```
print(X.isnull().sum())
```

```
Missing values:
```

```
radius1          0
texture1         0
perimeter1       0
area1            0
smoothness1     0
compactness1    0
concavity1      0
concave_points1 0
symmetry1       0
fractal_dimension1 0
radius2          0
texture2         0
perimeter2       0
```

```

area2                0
smoothness2         0
compactness2        0
concavity2          0
concave_points2     0
symmetry2           0
fractal_dimension2  0
radius3             0
texture3            0
perimeter3          0
area3               0
smoothness3         0
compactness3        0
concavity3          0
concave_points3     0
symmetry3           0
fractal_dimension3  0
dtype: int64

# Check duplicate rows
print("Number of duplicate rows:", X.duplicated().sum())

Number of duplicate rows: 0

# Check potential outliers using IQR
Q1 = X.quantile(0.25)
Q3 = X.quantile(0.75)

IQR = Q3 - Q1

outliers = ((X < (Q1 - 1.5 * IQR)) |
            (X > (Q3 + 1.5 * IQR)))

print("Potential outliers in each feature:")
print(outliers.sum())

Potential outliers in each feature:
radius1                14
texture1                7
perimeter1             13
area1                  25
smoothness1            6
compactness1           16
concavity1             18
concave_points1        10
symmetry1              15
fractal_dimension1     15
radius2                38
texture2               20
perimeter2             38

```

area2	65
smoothness2	30
compactness2	28
concavity2	22
concave_points2	19
symmetry2	27
fractal_dimension2	28
radius3	17
texture3	5
perimeter3	15
area3	35
smoothness3	7
compactness3	16
concavity3	12
concave_points3	0
symmetry3	23
fractal_dimension3	24

dtype: int64

```
import matplotlib.pyplot as plt
```

```
# Select key features
```

```
key_features = ['radius1', 'texture1', 'perimeter1', 'area1',  
                'smoothness1']
```

```
# Create histograms
```

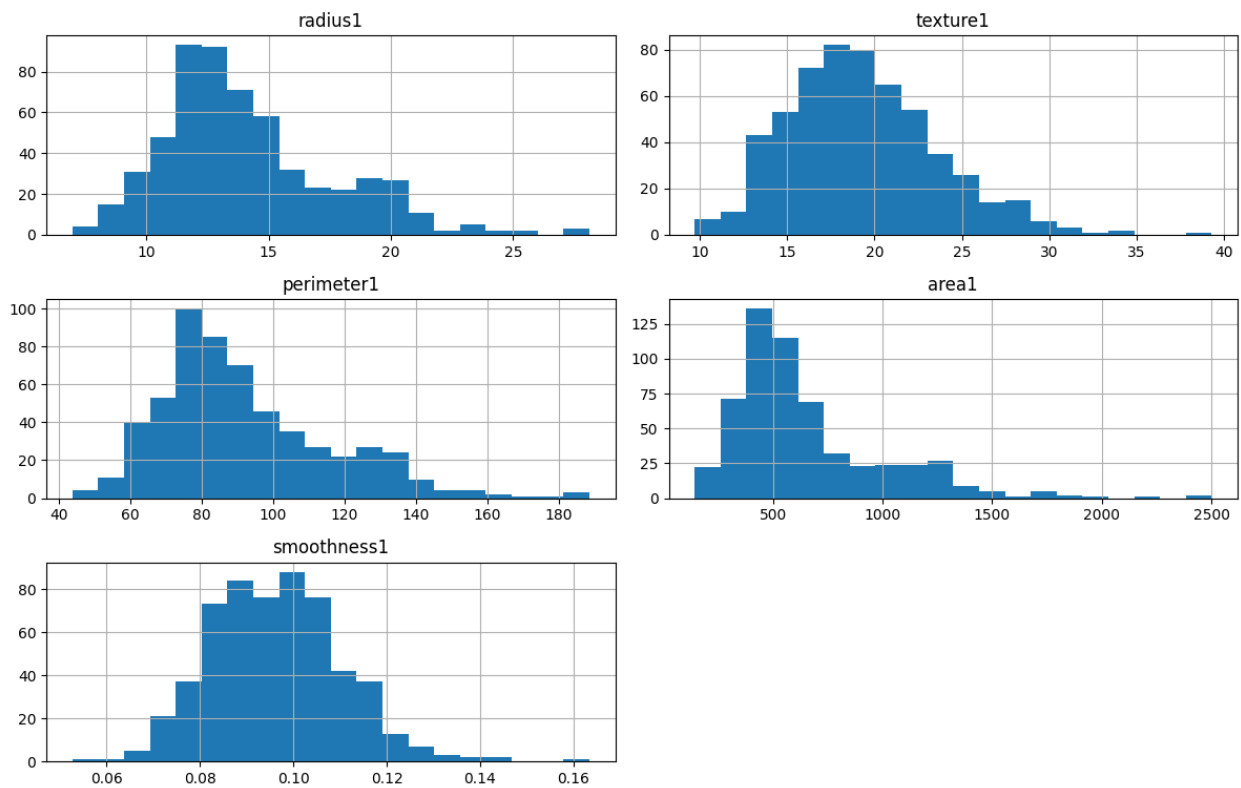
```
X[key_features].hist(figsize=(12, 8), bins=20)
```

```
plt.suptitle("Distribution of Key Features")
```

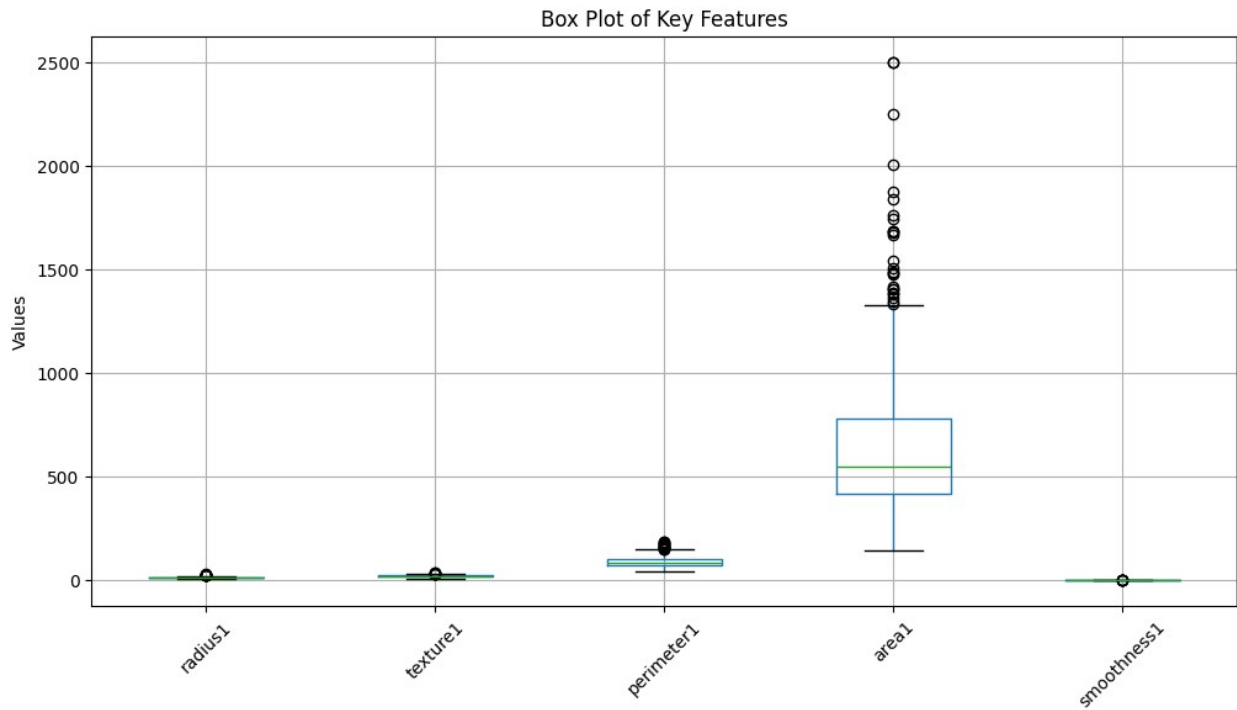
```
plt.tight_layout()
```

```
plt.show()
```

Distribution of Key Features



```
plt.figure(figsize=(12, 6))  
  
X[key_features].boxplot()  
  
plt.title("Box Plot of Key Features")  
plt.xticks(rotation=45)  
plt.ylabel("Values")  
plt.show()
```



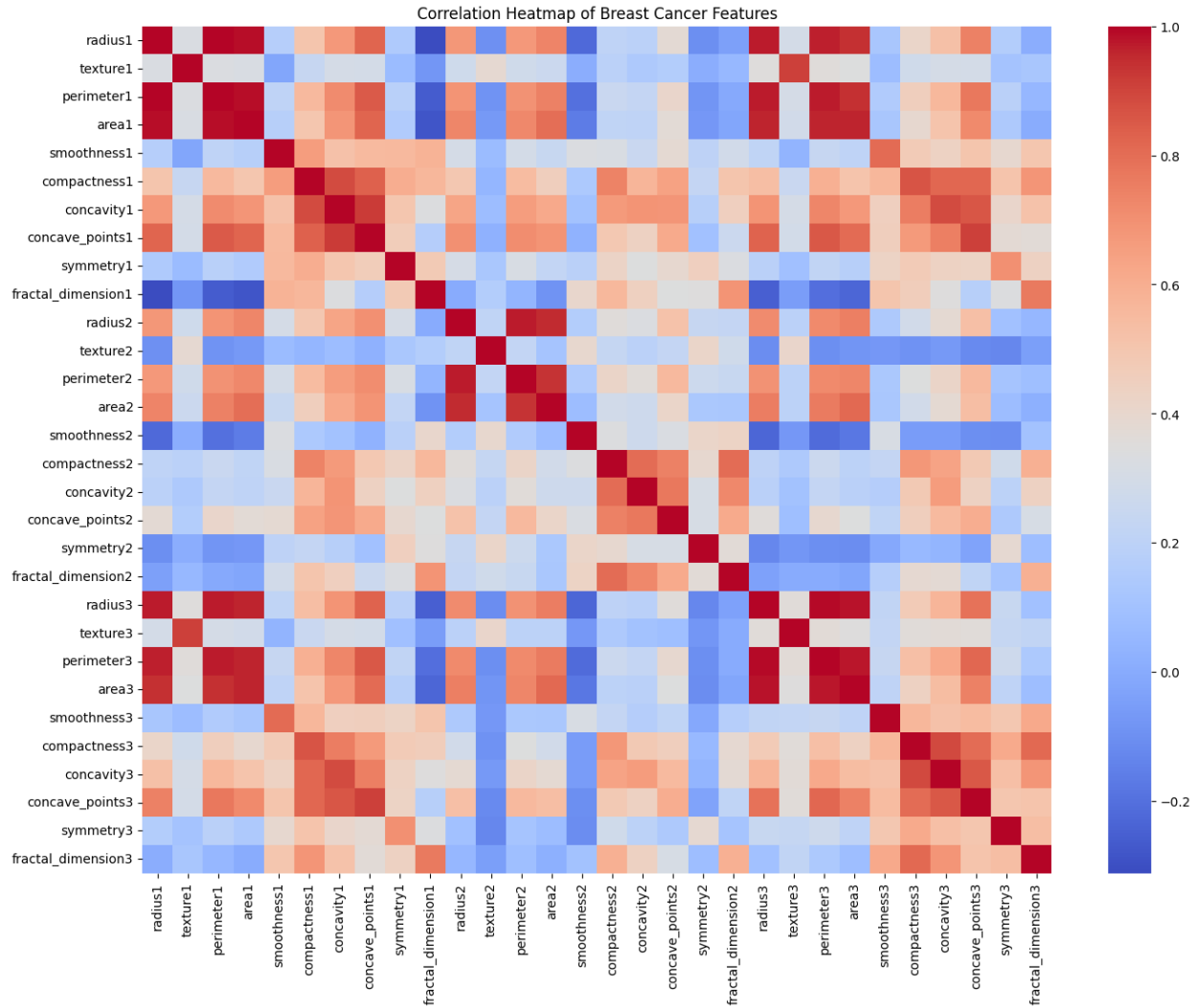
```
import seaborn as sns
import matplotlib.pyplot as plt

# Calculate correlation matrix
correlation_matrix = X.corr()

# Plot correlation heatmap
plt.figure(figsize=(16, 12))

sns.heatmap(
    correlation_matrix,
    cmap="coolwarm",
    annot=False
)

plt.title("Correlation Heatmap of Breast Cancer Features")
plt.show()
```



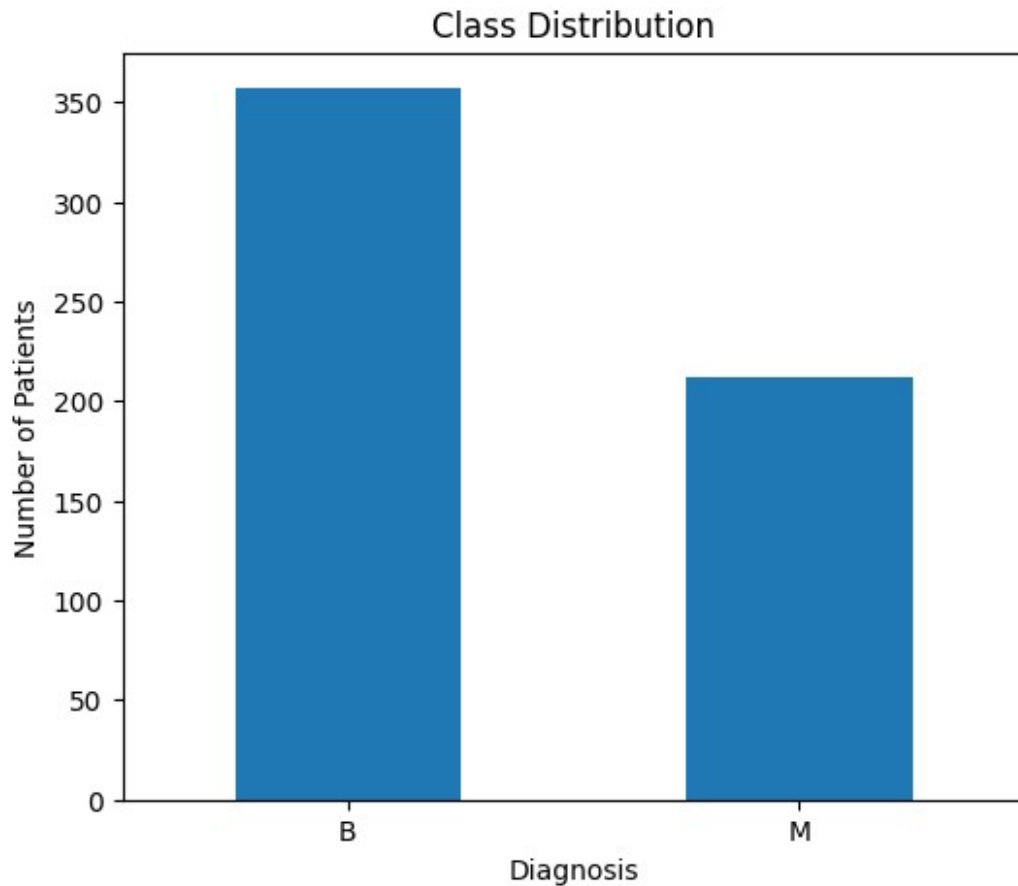
```
import matplotlib.pyplot as plt

# Count each class
print("Class distribution:")
print(y['Diagnosis'].value_counts())

# Visualize class distribution
y['Diagnosis'].value_counts().plot(
    kind='bar',
    figsize=(6, 5)
)

plt.title("Class Distribution")
plt.xlabel("Diagnosis")
plt.ylabel("Number of Patients")
plt.xticks(rotation=0)
plt.show()
```

```
Class distribution:
Diagnosis
B      357
M      212
Name: count, dtype: int64
```



```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# -----
# 1. Encode target variable as binary
# M = 1 (Malignant)
# B = 0 (Benign)
# -----

y_binary = y['Diagnosis'].map({'M': 1, 'B': 0})

print("Encoded target:")
print(y_binary.head())

print("\nClass distribution after encoding:")
print(y_binary.value_counts())
```

```

# -----
# 2. Split data into training and testing sets
# 80% training, 20% testing
# Stratified sampling maintains class proportion
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_binary,
    test_size=0.20,
    random_state=42,
    stratify=y_binary
)

print("\nTraining set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)

print("\nTraining class distribution:")
print(y_train.value_counts())

print("\nTesting class distribution:")
print(y_test.value_counts())

# -----
# 3. Standardize the features
# -----

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nScaled training data:")
print(X_train_scaled[:5])

print("\nScaled testing data:")
print(X_test_scaled[:5])

Encoded target:
0      1
1      1
2      1
3      1
4      1
Name: Diagnosis, dtype: int64

Class distribution after encoding:
Diagnosis

```

```
0    357
1    212
Name: count, dtype: int64
```

```
Training set shape: (455, 30)
Testing set shape: (114, 30)
```

```
Training class distribution:
Diagnosis
0    285
1    170
Name: count, dtype: int64
```

```
Testing class distribution:
Diagnosis
0    72
1    42
Name: count, dtype: int64
```

```
Scaled training data:
```

```
[[ 5.18558727e-01  8.91825791e-01  4.24631702e-01  3.83925436e-01
 -9.74743706e-01 -6.89771505e-01 -6.88586446e-01 -3.98175254e-01
 -1.03915470e+00 -8.25056321e-01 -1.09317755e-01 -5.59755400e-02
 -2.10096206e-01 -1.59132582e-02 -1.00518399e+00 -9.11941990e-01
 -6.62815884e-01 -6.52561081e-01 -7.01889114e-01 -2.75393571e-01
  5.79797697e-01  1.31324246e+00  4.66908134e-01  4.45982711e-01
 -5.96154777e-01 -6.34722227e-01 -6.10227299e-01 -2.35743918e-01
  5.45663235e-02  2.18367276e-02]
 [-5.16364088e-01 -1.63971029e+00 -5.41348716e-01 -5.42961327e-01
  4.76219058e-01 -6.31833818e-01 -6.04281166e-01 -3.03074908e-01
  5.21543093e-01 -4.54522896e-01 -6.04377961e-01 -1.00104604e+00
 -5.85429002e-01 -4.93453793e-01  4.03212009e-01 -7.68173276e-01
 -4.79187222e-01  1.14508478e-01 -1.42950761e-01 -5.77397732e-01
 -5.82458953e-01 -1.69029101e+00 -6.11934288e-01 -5.87013537e-01
  2.73581959e-01 -8.14844486e-01 -7.12666415e-01 -3.23207881e-01
 -1.37576237e-01 -9.04401643e-01]
 [-3.68118388e-01  4.55514964e-01 -3.88249933e-01 -4.02970039e-01
 -1.43297929e+00 -3.83927370e-01 -3.42175070e-01 -7.65459348e-01
 -8.50857052e-01 -2.26170901e-01  3.03979894e-01  1.05150064e+00
 -1.69545176e-01 -8.09073190e-04 -3.10104091e-01  1.10632991e+00
  6.22584752e-01  2.73684564e-01  7.54482587e-01  1.50810455e+00
 -3.98622224e-01  1.81977388e-01 -4.75431283e-01 -4.20778328e-01
 -1.62278520e+00 -3.91399175e-01 -4.31312898e-01 -8.90825037e-01
 -6.75892999e-01 -1.44015592e-01]
 [ 2.05284794e-01  7.26167669e-01  4.00330308e-01  7.06116009e-02
  2.43253484e-01  2.20358457e+00  2.25609405e+00  1.21323326e+00
  8.18473995e-01  8.99791136e-01 -5.45729713e-01 -6.21676545e-01
  2.61427407e-01 -3.53584902e-01  2.44600975e-02  2.09072809e+00
  1.49056147e+00  1.69512701e+00 -6.54909382e-01  7.67547750e-01
 -3.09312591e-04  2.74191135e-01  5.13776118e-01 -9.94819409e-02]
```

```
4.18538082e-01 2.86596968e+00 2.95861933e+00 1.97706439e+00
-7.56459905e-02 1.72884831e+00]
[ 1.24300470e+00 1.94195111e-01 1.21037678e+00 1.20665201e+00
-1.11441850e-01 5.13480707e-02 7.32962322e-01 7.13767250e-01
-4.27187350e-01 -8.22183969e-01 1.52386344e+00 1.14394136e+00
1.28274755e+00 1.08131268e+00 1.35184527e+00 1.07730938e-01
5.92926514e-01 1.18098825e+00 3.01549783e-01 1.71526944e-01
1.01283532e+00 2.23144239e-01 9.38517229e-01 8.80910405e-01
7.32014364e-02 -2.77005741e-01 3.27775115e-01 5.01858851e-01
-9.09322392e-01 -5.46248793e-01]]
```

Scaled testing data:

```
[[ -0.77089916 -2.00602473 -0.76451652 -0.71183971 -0.1583148 -
0.68680034
-0.66248612 -0.57475415 -0.53582061 -0.22760708 -0.93280815 -
1.37244026
-0.85278289 -0.6368401 -0.2999339 -0.56993036 -0.53250895 -
0.83258923
-0.8416237 -0.59337276 -0.72135781 -1.63595077 -0.70654903 -
0.65337009
0.96851866 -0.10446758 -0.30668651 -0.38805702 0.16254727
0.06095804]
[ 1.89472636 0.96648861 1.89081582 1.95660534 0.32930383
1.05411572
2.23036133 2.06812735 1.4123358 -0.53638493 2.04514286 -
0.71973622
1.63167503 1.99915055 -0.56541094 0.78141602 1.99285843
1.10617549
0.4316475 0.47010788 1.88503847 0.18033071 1.68254649
1.92439305
-0.46825232 0.38976262 2.00126206 1.42292627 0.33722232 -
0.31041892]
[ 0.56051506 -0.78108791 0.57044007 0.35809371 0.19638053
0.74214357
-0.27784328 0.12500278 0.63379746 0.43159774 -0.81654662 -
1.32114472
-0.73678808 -0.54559427 -0.83649911 -0.40917752 -0.39462969
0.04287924
-0.13090468 -0.02663951 0.12633377 -1.11395509 0.15347849 -
0.04997239
-0.36593035 -0.00334631 -0.30102167 0.14391558 0.38009711
0.30174696]
[ 0.07102454 0.08220089 0.0917026 -0.04743661 0.1068322
0.19062136
0.05753979 0.23649973 0.23185441 0.17021369 -0.41980848
1.14756649
-0.25253333 -0.29854773 0.38918416 0.20600035 0.03762334
0.40580072
-0.6994799 0.26091105 -0.00439457 0.8159469 0.04802552 -
```

```

0.11031216
  0.46969907  0.29748946  0.20881356  0.61418415 -0.31066334
0.51829168]
[-0.21987269  2.6370691 -0.23798631 -0.28436631 -0.24786313 -
0.54919834
 -0.74740408 -0.41305833 -1.59318431 -0.36691616 -0.24282877
1.27988361
 -0.29449893 -0.25551115 -0.36972245 -0.73068321 -0.75980463 -
0.40790744
 -0.57660983 -0.29707397 -0.26585125  2.5861215 -0.32721128 -
0.31797724
 -0.63452552 -0.71688326 -0.96838655 -0.56702865 -1.13957587 -
0.40629368]]

```

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    roc_curve,
    roc_auc_score
)
import matplotlib.pyplot as plt

```

```

# -----
# 1. Train Logistic Regression Model
# -----

```

```

model = LogisticRegression(random_state=42)
model.fit(X_train_scaled, y_train)

```

```

# -----
# 2. Make Predictions
# -----

```

```

y_pred = model.predict(X_test_scaled)

# Probability of malignant class (1)
y_prob = model.predict_proba(X_test_scaled)[: , 1]

```

```

# -----
# 3. Accuracy

```

```

# -----

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

# -----
# 4. Precision, Recall and F1-score
# -----

precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("\nPrecision:", precision)
print("Recall:", recall)
print("F1-score:", f1)

# -----
# 5. Classification Report for Both Classes
# -----

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Benign", "Malignant"]
    )
)

# -----
# 6. Confusion Matrix
# -----

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Visualize confusion matrix
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Benign", "Malignant"]
)

disp.plot()
plt.title("Confusion Matrix - Logistic Regression")

```

```

plt.show()

# -----
# 7. ROC Curve and AUC
# -----

fpr, tpr, thresholds = roc_curve(y_test, y_prob)
auc_score = roc_auc_score(y_test, y_prob)
print("\nAUC Score:", auc_score)

# Plot ROC curve
plt.figure(figsize=(7, 5))

plt.plot(
    fpr,
    tpr,
    label=f"Logistic Regression (AUC = {auc_score:.3f})"
)

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

```

Accuracy: 0.9649122807017544

Precision: 0.975

Recall: 0.9285714285714286

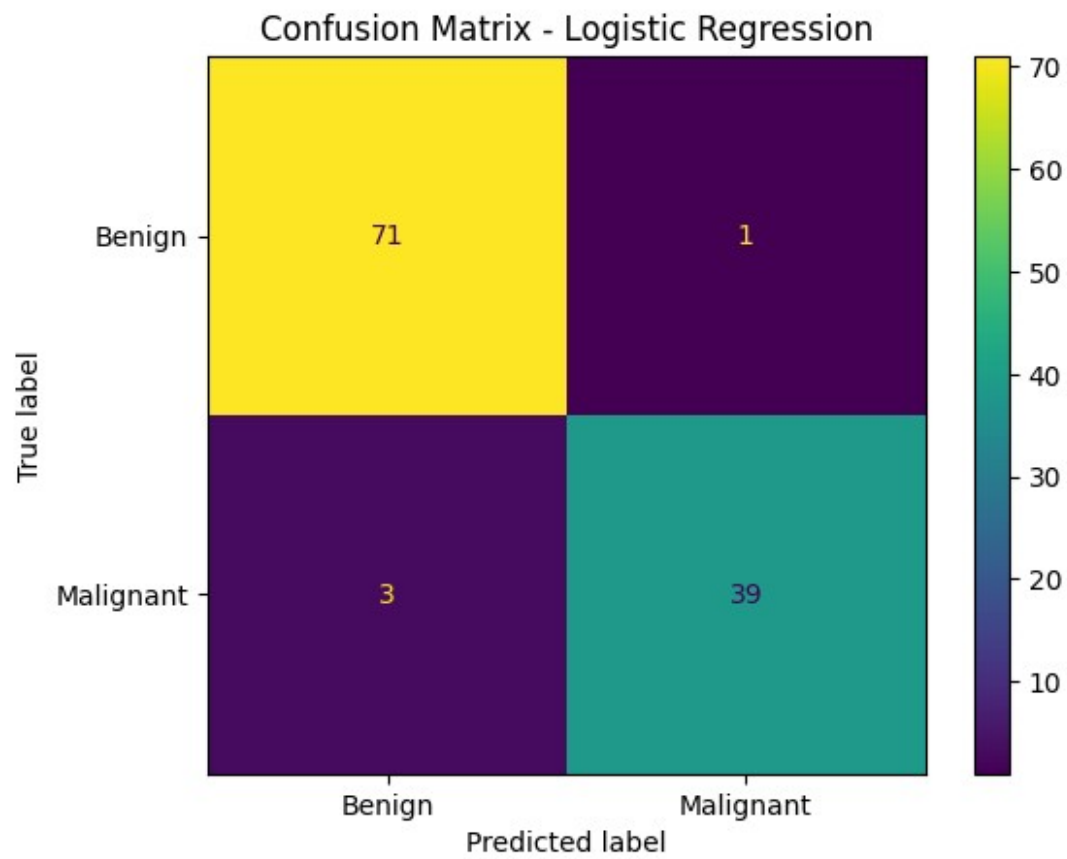
F1-score: 0.9512195121951219

Classification Report:

	precision	recall	f1-score	support
Benign	0.96	0.99	0.97	72
Malignant	0.97	0.93	0.95	42
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

Confusion Matrix:

```
[[71  1]
 [ 3 39]]
```



AUC Score: 0.996031746031746

